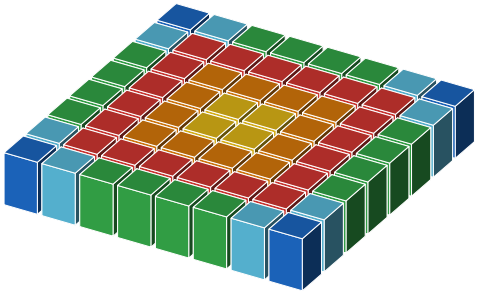
The 16GB Ceiling

Test-Time Training for ARC-AGI-2 on One Consumer GPU

Running the 2025 ARC Prize winning recipe, per-task test-time training
with augment-infer-reverse-vote selection, on one 16 GB card



16.28%

on a single 16 GB GPU

up from 12.79%, 13× faster

ARC Prize 2026 · Paper Track

ARC Prize 2026 Paper Track entry

Open source under MIT-0

Abstract

We run the 2025 ARC Prize winning recipe, a small grid model with per-task test-time training (TTT) and augment-infer-reverse-vote (AIRV) selection, on a single 16 GB consumer GPU, and ask what actually binds at that scale. The binding constraint is generation throughput, not model quality and not memory. Moving decode off the Hugging Face eager path onto llama.cpp buys a 13× speedup, turns an overnight failure-prone evaluation into a sub-90-minute one, and is what makes per-task TTT tractable at all, by hot-swapping each task's adapter on the fast engine. Building on the NVARC Qwen3-4B grid model, the system scores **12.79%** without TTT on the 120-task public evaluation set. The lift from TTT depends on regularization. A naive adapter overfits and ties the baseline, while a regularized, scale-tuned configuration reaches **17.6%** on the 84 trainable tasks and lifts the full evaluation to a blended **16.28%**. On the hidden main-track leaderboard the same recipe scores **13.61%** with TTT and 6.53% without, and the public-to-hidden transfer is itself a finding. The no-TTT baseline halves, and across the follow-on submissions every candidate-side lever that lifted or tied the public evaluation regressed the hidden score, so public-eval gains must be re-validated in the exact deployed condition before they are trusted. The climb then came from the training side rather than the vote. Deepening the regularized adapter lifted the hidden score to **18.19%**, a 4.58-point gain and the first configuration to beat the shipped operating point. The number is not a wall but climbs under the 16 GB ceiling as the per-task training deepens, while the methods that reach the field's level, heavy rank-256 test-time training at an eight-thousand-token context, run far slower than the contest budget allows on one card. That configuration turns out to fit the card's memory once the attention kernel is chosen deliberately, which corrects an earlier conclusion of our own, so the binding resource is time rather than capacity. We then measure the gap rather than assert it. Reproducing one such public pipeline on the contest's own four-L4 node, and repairing two defects the community port had introduced into it, scores **30.97%**, a score roughly sixty other entries carry exactly because it is the public notebook's own number. That result is a reproduction of named public work and is not a single-card result, and we report it as the measured price of the constraint, roughly twelve points against the best configuration we could serve inside the single-card budget. We give an honest accounting of what the recipe costs and where the effort should go for anyone working with one consumer GPU rather than the winner's hardware. The contest itself provides four L4 GPUs, 96 GB together, where the offline frontier now leads near 66% on the strength of large-scale synthetic-data pretraining, so we read the 16 GB result as the efficient floor of a larger climb, and the ceiling in our title is the 16 GB card itself, the hardware wall that shapes every choice here and not a limit on the task. All code, the conversion recipe, and a development log of every measurement are open source under MIT-0.

Contents

1 Introduction **3**

1.1 Efficient abstraction is the point 3

1.2 Contributions 4

2 Related Work **4**

3 System and Method **5**

3.1 Model and prompting . . . 5
3.2 The engine is the bottleneck . . . 5
3.3 AIRV: augment, infer, reverse, vote . . . 6
3.4 Per-task test-time training . . . 7

4 Results **8**

4.1 Two regimes, and the leaderboard progression . . . 8
4.2 The engine speedup . . . 12
4.3 Test-time training . . . 13
4.4 Ablations on scale, colour, candidates, filtering, and the big-grid wall . . . 14

5 Discussion **15**

5.1 Four lessons that generalize . . . 15
5.2 Accuracy per GPU-hour . . . 16
5.3 Limitations and ongoing work . . . 16
5.4 Scaling to the full contest hardware . . . 18
5.5 A routed dual-arm system . . . 19
5.6 A stronger public decoder, and its coupling to training . . . 20
5.7 The selection leak, measured on the candidate pool . . . 22
5.8 Revisiting the big-grid wall with the host's memory . . . 25

6 Conclusion **26**

A Reproducibility **27**

B Extended results **28**

C Engineering log of walls and dead ends **30**

D Prompt and serialization format **30**

E Data and model provenance **31**

Reader's guide. Section 1 motivates efficient ARC and states the contributions. Section 2 places the work against the 2025 winning systems. Section 3 is the method, covering the model and prompt format (3.1), the engine bottleneck and its fix (3.2), the AIRV selection rule (3.3), and the per-task TTT bridge (3.4). Section 4 reports the numbers. Section 5 draws the lessons. Readers who want to reproduce the system should go straight to Appendix A. The per-scale tables and significance detail are in Appendix B, the honest list of dead ends in Appendix C, the exact prompt and 16-token vocabulary in Appendix D, and

data licensing and BibTeX in Appendix E.

1 Introduction

1.1 Efficient abstraction is the point

ARC-AGI-2 [1, 2] is a benchmark of abstraction and reasoning. Each task gives a few input/output grid pairs that demonstrate a transformation, and the solver must apply that transformation to a held-out input. The tasks are designed to be easy for humans and hard for machines, and the competition is run *offline*, which rules out frontier API models and forces every entrant to ship a self-contained system. The 2025 ARC Prize was won by NVARC at 24.0% on ARC-AGI-2 using a 4B grid model with per-task TTT [3, 4]. The same recipe reproduced across the top teams. They fine-tune a small model on augmented grid data, train a fresh low-rank adapter on each test task, then sample many augmented candidates and vote.

The recipe is effective but front-loaded with compute. The base models are pre-trained on clusters of H100 or GH200 GPUs, though the winning offline submission itself runs on four L4 GPUs. Most researchers do not have that hardware, and the ARC Prize's stated mission is *efficient* skill acquisition, not skill bought with compute. We therefore characterize the recipe under a constraint we think is widely shared, a single RTX 5070 Ti with 16 GB, on the Blackwell architecture, running native Windows. The aim on one card is not to top the leaderboard but to build a working, reproducible system, measure honest numbers, and map where the small model stops being enough. That map is the efficient floor of a larger climb. The ceiling in the title is the hardware, 16 GB of VRAM, not a score, and the number climbs beneath it as the method deepens, from 13.61 to 18.19 on the hidden set (Section 4). We also put a number on the ceiling itself. Reproducing a public pipeline that cannot fit one card, on the hardware the contest provides, reaches 30.97% on the same hidden set, so the constraint is worth roughly twelve points rather than an unknown amount. The same contest hands every entrant four L4 GPUs, 96 GB together, and the offline frontier on that hardware has already passed 50% with refinement loops and program synthesis [3]. We read the 16 GB walls not as the limit of the task but as the exact places scaling up has to break, and Section 5.4 lays out that path. Figure 1 shows a representative task our system solves end to end.

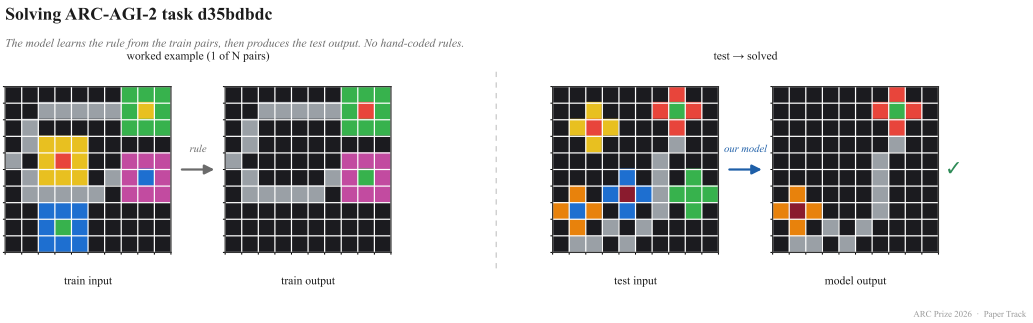


Figure 1: A real ARC-AGI-2 evaluation task solved by the no-TTT pipeline, shown as colour grids. The few-shot train pairs establish the rule, and the model produces the held-out test output exactly. Grids are rendered with the model's own 16-colour serialization (Appendix D).

1.2 Contributions

1. A reproducible offline ARC-AGI-2 system for the custom-vocabulary NVARC model on a single 16 GB card, with a clean **12.79%** no-TTT baseline (Section 4, Appendix A).
2. The finding that, on this hardware class, the inference *engine* dominates every other knob, since moving decode off the Hugging Face eager path onto llama.cpp is worth a 13× speedup (Section 3.2).
3. A practical bridge that makes per-task TTT runnable on one card by hot-swapping GGUF adapters on the fast engine, and a measured, regularization-dependent lift to a **16.28%** blend (Section 3.4, 4.3).
4. An "accuracy per GPU-hour" accounting that names the three walls a compute-poor practitioner hits first, the decode ceiling, the VRAM limit on large-grid TTT, and adapter overfitting (Section 5).
5. The measured finding that public-eval gains do not transfer to the hidden set. The no-TTT baseline halves from 12.79% to 6.53%, and across seven leaderboard submissions every candidate-side lever that lifted or tied the public evaluation regressed the hidden score when deployed (Section 4, 5.1).
6. A reading of why those walls are a property of a small model on one card rather than of the task, and a concrete path that scales the same verifier, TTT, and selection onto the full 96 GB the contest provides, where the offline frontier now leads near 66% (Section 5.4).
7. A complete open-source release, the conversion patches, the pipeline, the TTT harness, and a development log of every measurement and dead end behind these numbers (Appendix C).

2 Related Work

ARC and the measure of intelligence. ARC was introduced to measure skill-acquisition efficiency rather than skill itself [1]. ARC-AGI-2 [2] raises compositional difficulty while keeping tasks human-solvable, widening the human/machine gap that frontier models had begun to close on ARC-AGI-1.

Test-time training. The dominant 2025 approach fits a small per-task adapter at inference, trading test-time compute for generalization. The ARC Prize 2025 technical report [3] identifies this, together with heavy synthetic pre-training and disciplined engineering, as the core of the winning NVARC solution [4], whose released model we build on directly. We hold this recipe fixed and ask what survives a single consumer GPU.

Augmentation and voting. Predicting under the eight dihedral symmetries of the grid, inverting each prediction back to the original frame, and voting is a standard self-consistency device for ARC. It is cheap, training-free, and provides a usable confidence signal (Section 3.3).

Synthetic data. Procedural generators such as RE-ARC [6] and LLM-mutated corpora supply the augmented grid data these models are trained on. We use RE-ARC only in a base-model probe (Section 5.3), not in the results below.

Efficient and on-device inference. Our engineering contribution, getting a custom-vocabulary grid transformer to decode near the memory-bandwidth roofline on a consumer card and then serving per-task adapters on it, sits in the practical lineage of quantized GGUF inference. The novelty is not the engine

but the demonstration that the full TTT recipe, adapter serving included, fits one 16 GB card, and a measurement of exactly where it stops.

3 System and Method

3.1 Model and prompting

We use the NVARC `qwen3_4b_grids15_sft139` model, a Qwen3-4B [5] with a 16-token vocabulary, the ten grid colours as digits, a newline, two role markers, and three specials (Appendix D). A grid is serialized one token per cell, and a colour value equals its token id. Because the role and special tokens cannot be produced by tokenizing text, we assemble every prompt at the token-id level as a ChatML few-shot sequence. Getting this byte-exact is the single most important correctness condition, since a one-token drift between training and inference silently destroys accuracy.

3.2 The engine is the bottleneck

Our most consequential finding concerns throughput. Decoding a 4B model is memory-bound. Each token streams the full weight matrix from VRAM, so for a weight footprint M on a card of bandwidth B the decode rate is bounded by a first-order roofline

$$r_{\max} = \frac{B}{M}. \tag{1}$$

With $B = 896$ GB/s, Eq. (1) gives 132 tok/s in bf16 ($M = 6.77$ GB), 249 in Q8_0 ($M = 3.6$ GB), and 444 in Q4_K_M ($M = 2.02$ GB). The standard Hugging Face eager path prefills healthily ($\approx$ 3,400 tok/s) but *decodes* at only $\approx$ 13 tok/s, about a tenth of the bf16 roof, because of per-token framework overhead with no CUDA graphs. The usual remedy, compiling the decode step, is unavailable on this platform. `torch.compile` needs Triton, and Triton has no native-Windows backend (Appendix C).

The fix is to change engines. We convert the model to GGUF and serve it with llama.cpp. The custom vocabulary needs two small, documented converter patches (Appendix A): the unknown tokenizer hash is mapped to the `qwen2` pre-tokenizer it actually uses, and two throwaway byte-pair merges are added to satisfy a loader that requires a non-empty merge list (they never fire on grid text). The prebuilt CUDA 12.4 binary runs on the Blackwell card through PTX just-in-time compilation. Measured decode rises to 102 tok/s in bf16, 170 in Q8_0, and 244 in Q4_K_M (Figure 2), a 13$\times$ speedup over eager at Q8_0, which we adopt. It is near-lossless (14 of 15 training tasks reproduced exactly under greedy decoding) and has a 3.6 GB footprint. The server's default prompt cache needs one note. It reserves $\approx$ 680 MiB per unique prompt, and since every ARC prompt is unique it yields no reuse and destabilizes sustained generation, so we disable it and run a single supervised stream.

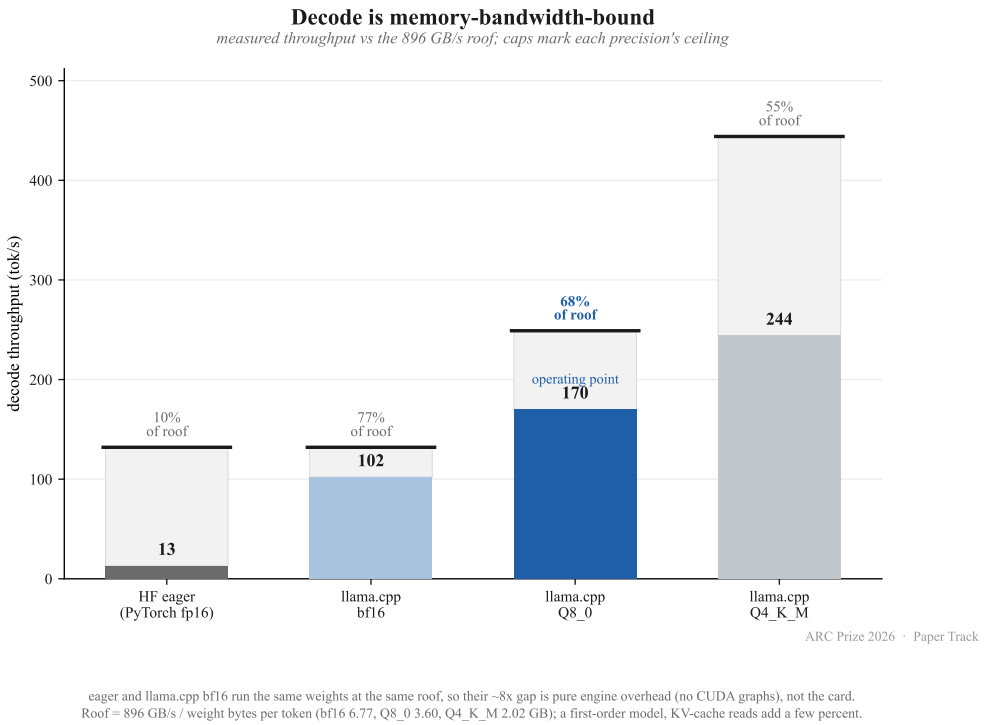


Figure 2: Decode throughput versus the memory-bandwidth roofline of Eq. (1) (the ceiling is a first-order model, bytes/token $\approx$ weight bytes). Bars are *measured* tok/s; the shaded gap to each cap is framework overhead, not the card. The eager path fills only $\approx 10\%$ of its roof, and moving decode to llama.cpp at Q8_0 recovers the $13\times$.

3.3 AIRV: augment, infer, reverse, vote

Let f be the served model and $\mathscr{G}$ the eight dihedral (D4) symmetries of the grid. For a test input x we generate one prediction per symmetry, invert it back to the original frame, and vote. Writing n_c for a candidate's vote count and $\bar{\ell}_c$ for its mean sequence log-prob,

$$c_g = g^{-1}\big(f(g(x))\big), \qquad \hat{Y} = \operatorname*{select}_{c}\,\big(n_c,\,\bar{\ell}_c\big), \qquad n_c = \big|\{g \in \mathscr{G} : c_g = c\}\big|, \quad \bar{\ell}_c = \frac{1}{n_c}\sum_{g:\,c_g=c} \log p(c_g), \tag{2}$$

where select returns the two distinct candidates with the highest vote count, ties broken by mean log-prob, and emits them as the two attempts the metric allows. The vote count is itself a usable confidence signal. Agreement across symmetries predicts correctness (Figure 6). The whole offline system (the trainer, the converter, and the server, which never share VRAM) is sketched in Figure 3.

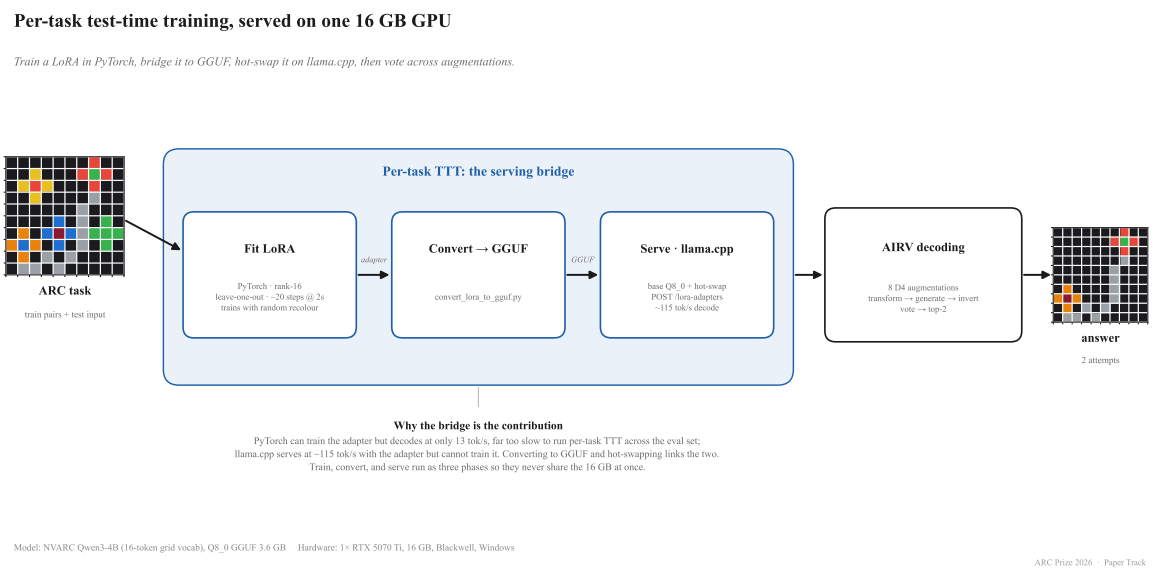


Figure 3: The offline system. The PyTorch trainer and the llama.cpp server never share VRAM; per-task adapters cross the bridge as GGUF and are hot-swapped on the fast engine per request.

3.4 Per-task test-time training

For TTT we fit a per-task LoRA adapter $\Delta W = sBA$ of rank r and scale s on the task's own train pairs T, with a leave-one-out target and the loss masked to the answer tokens:

$$\mathscr{L}(\theta) \;=\; -\sum_{(u,v)\in T}\;\sum_{t}\log p_\theta\big(v_t \mid v_{<t},\, \mathrm{ctx}_{T\setminus\{(u,v)\}},\, u\big), \tag{3}$$

drawing a fresh random augmentation each step. Capping the few-shot context to $\approx 1{,}600$ tokens drops a training step from ≈ 13 s to ≈ 2 s, so training is no longer the expensive part.

The harder problem is *serving* the trained adapter. TTT must run in PyTorch, where decoding is the slow 13 tok/s and intermittently becomes quadratic after a step. The fast engine does not train. We bridge them (Figure 3): train in PyTorch, save each adapter, convert it to a GGUF adapter, and hot-swap it on llama.cpp through its `/lora-adapters` endpoint, where decode holds ≈ 115 tok/s with no stalls. We run three phases (train all, convert all from a clean process, then serve and generate all) so the two frameworks never share VRAM (Figure 4). Large-grid tasks remain too VRAM-bound to train and are excluded by their largest-train-pair token length (Figure 4).

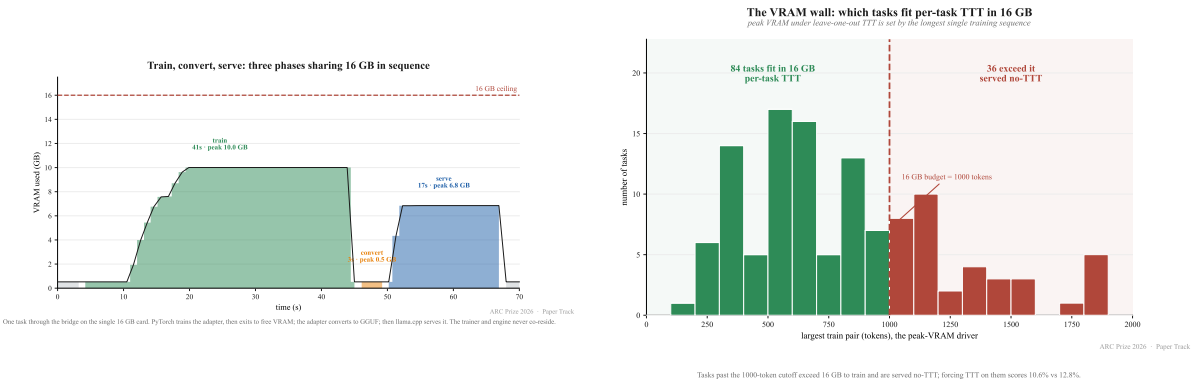


Figure 4: The left panel shows the three-phase schedule keeping PyTorch and llama.cpp from sharing the 16 GB at once, with the VRAM trace sampled live and the peaks measured. The right panel shows the VRAM wall, with each task placed by grid size and context length, and tasks above the operational cutoff excluded from TTT. In the right panel only the per-task points and the cutoff are measured. The translucent VRAM surface is a calibrated model, not a measurement, drawn only to make the wall legible, and the cutoff metric is the largest train pair.

4 Results

4.1 Two regimes, and the leaderboard progression

Two hardware regimes appear in this paper, and every number belongs to exactly one of them. Every public-eval number, the 12.79% baseline, the 17.6% trainable-task peak, and the 16.28% blend, is measured on the single RTX 5070 Ti against the 120-task public evaluation set, and all method development ran there. Every leaderboard number is measured by Kaggle's offline scoring rerun on the contest's four-L4 node, against the hidden 240-task set (120 semi-private plus 120 private). The submissions through v6.1 use the four L4s only to shard tasks for throughput, since the bf16 per-task training does not fit the 12-hour budget on one card. No single task there needs more than one 16 GB GPU, so the method under test is the single-card recipe. The v7 entries are the one exception and this paper marks them as such wherever they appear. They reproduce a public pipeline whose per-task training runs at rank 256 over an eight-thousand-token context, which fits 16 GB only under an attention kernel we identified after those runs and would still need roughly forty hours on one card against the twelve the contest allows, so those numbers measure competition hardware rather than the card this work is about. They are kept in the same table deliberately, because the distance between the best single-card entry and the best four-L4 entry is the most direct measurement we have of what the constraint costs.

That exclusion is measured rather than assumed. Sweeping LoRA rank against sequence length on the single card, with training configured exactly as our shipped path, every rank from 32 to 256 fits comfortably at a two-thousand-token sequence and every one of them fails at four thousand. The blocking term is neither the adapter nor the optimizer, which together account for under a gigabyte at rank 32 and three at rank 256. It is activation memory, which grows from roughly 2.6 GB at two thousand tokens to roughly 8.5 GB at four thousand, a factor of 3.3 for a doubling of length. That superlinear growth is attention materializing its score matrix. Leaner adapter dtypes buy a full rank doubling at equal memory,

enough to run rank 256 at two thousand tokens in 14.5 GB, but they leave the activation term untouched. On that evidence we recorded the context as a hard wall and the eight-thousand-token configuration as beyond the card.

That conclusion was wrong, and the sweep above contains the reason. It never chose an attention kernel. Repeating the measurement with the backend forced rather than left to the framework's dispatch (Table 1) moves the wall entirely. Under cuDNN attention the rank-256 configuration trains at an eight-thousand-token sequence in **14.86 GB** at 5.04 seconds per step, on the same card that had failed at four thousand. The framework's automatic selection is numerically identical to the forced math backend at both lengths, because the flash and memory-efficient kernels report no available implementation for this GPU in this build, so the dispatcher had been falling back to the quadratic path throughout. The result is corroborated from outside the measurement, since the four-L4 runs fit the same training configuration into 13.0 GB under a flash-attention kernel, and two different memory-efficient implementations on two different cards landing within two gigabytes of each other is the agreement we would expect.

Three limits on that result, stated because they bound what follows from it. It measures training memory for a single sequence at batch size one, where the deployed pipeline may pack differently. It measures memory and step time rather than numerical equivalence, which would need a paired run against the math backend on a real task. And it shows only that the training configuration fits, not that the pipeline runs end to end on one card, which would additionally require porting that pipeline off a training library with no working install on this platform.

What it does settle is which resource binds. At 5.04 seconds per step the 128-step schedule is about eleven minutes for one adapter, so the 240-task contest set would need roughly forty hours against a twelve-hour budget. The card holds the winning configuration and cannot serve it in time. That is a throughput limit rather than a capacity limit, and it is the second time in this work that a ceiling we recorded turned out to be a default rather than a property of the hardware, after the six-hundred-token training truncation of Section 5.8. Both were found by re-examining a measurement we had already filed as settled.

Table 1: Attention backend against sequence length, at rank 256 with bf16 adapters on the 16 GB card, peak reserved memory in GB. Cells marked thrash spilled into host memory rather than raising, running at 33 to 55 seconds per step against roughly 2 seconds when resident. Blank cells were not attempted once the shorter sequence had failed.

attention backend	2048	4096	8192
eager	13.44	19.33 (thrash)	
sdpa, framework default	13.94	18.62 (thrash)	
math, forced	13.94	18.62 (thrash)	
memory-efficient, forced	no kernel available for this GPU		
flash, forced	no kernel available for this GPU		
cuDNN, forced	**12.31**	**12.80**	**14.86**

The AIRV pipeline without TTT scores **12.79%** (22 of 172 test outputs) on the public evaluation, a clean single-stream run completing in $\approx$ 78 minutes, and **6.53%** on the hidden set, a halving we return to in

Sections 5.1 and 5.3. Adding per-task TTT roughly doubles the hidden score. Table 2 gives the full progression.

Table 2: Leaderboard submissions, scored on the hidden 240-task set on the four-L4 node. Entries through v6.1 are the single-card recipe sharded for throughput. The v7 entries reproduce a public pipeline whose per-task training cannot be served in budget on one 16 GB card and are not single-card results.

version	configuration	hidden score (%)
v1	no-TTT AIRV, eight greedy D4 views	6.53
v2	+ per-task TTT (160 tasks) + colour-perm	11.94
v2.1	scaled TTT (232 tasks), single scale, no colour-perm	**13.61**
v3	+ three-scale vote	error
v3.1	+ two-scale vote + colour-perm	12.78
v4	+ more-candidates sampling (four per view)	13.19
v5	+ white-box plausibility filter	12.78
v6	deep-regularized TTT (60 steps, rank 32), disk-exhausted	12.36
v6.1	deep-regularized TTT, all 232 tasks trained	**18.19**
v7	sorokin reproduction as ported (Section 5.6), not servable in budget on one card	16.81
v7.1	same pipeline with both port defects repaired	30.56
v7.2	identical build, cap knob inert in both, a repeat run	**30.97**
v7.3	+ loss-gated search skip, greedy fill of empty slots	28.61

The scaled single-scale configuration (v2.1) reached **13.61%**, ranking 468 of about 4000, and four candidate-side submissions failed to beat it. One errored in the scored rerun, and each of the three that scored carried exactly one lever that had lifted or tied the public evaluation, multi-scale voting with colour permutation, more-candidates sampling, and the white-box plausibility filter, and each regressed the hidden score (Section 4.4 gives the levers and the mechanism). The eight greedy D4 views at a single adapter scale are a local optimum that the candidate and voting side does not improve, and the repeated public-to-hidden reversal is itself a finding (Section 5.1). The break came from the other side of the pipeline. A deep-regularized adapter that trains longer at a higher rank (60 steps, rank 32, Table 9), moving the per-task training rather than the vote, lifted the hidden score to **18.19%** in v6.1, a 4.58-point gain and the first configuration to beat v2.1. Its intermediate form v6 scored 12.36% as the same recipe crippled by a disk-exhaustion bug that left 73 of 232 tasks with no adapter, and fixing the scratch path recovered the full lift. This is the paper's thesis made concrete, the candidate and voting side is saturated and the per-task training side is where the hidden number moves. For reference the 2025 winner reached 24.0% with TTT [3] on the same Qwen3-4B base lineage with a more developed recipe.

The two v7 entries then measure the constraint itself. Reproducing the public pipeline of Section 5.6 on the four-L4 node scored 16.81% as the community port shipped it, and **30.56%** once two defects introduced by that port were repaired, and **30.97%** when the same repaired pipeline ran at the upstream per-task cap. The last of those is the highest this project has posted and it is not a single-card result, since its per-task training cannot be served on one card inside the contest's twelve hours. Read against v6.1, the best configuration we ran within that budget, the pair gives the cleanest figure in this work. The same

author, the same base model, the same hidden set and the same twelve-hour budget yield **18.19%** under 16 GB and **30.97%** on the contest's 96 GB, so roughly twelve points separate what one consumer card reaches from what the provided hardware reaches on a method built for it. That gap is the honest size of the ceiling in the title, measured rather than argued, and it is the paper's central quantitative claim.

What the pair does not do is attribute that gap. The two runs differ in their hardware and in their method at the same time, so twelve points is the size of the distance rather than an account of where it comes from. Isolating the method term would mean running the reduced configuration on the four-L4 node rather than on the single card, holding the platform fixed and moving only the training context, and we flag that as the experiment this measurement wants next. Running it on our own hardware instead would confound the answer with the card, the driver and the attention kernel all at once, which is the fault the sweep in Section 4 corrected in itself.

A third submission of the reproduction, identical to the second except that it leaves the per-task search cap at the upstream default of 1800 seconds rather than the 1200 we had chosen for margin, scored **30.97%** and placed 264th. Two facts about that number matter more than its size.

Roughly sixty entries on the leaderboard carry it exactly. A tie that wide is not coincidence, it is the public notebook being forked and submitted unchanged, which identifies 30.97 as that notebook's number rather than as anyone's result. Our restoration therefore converges on the public baseline exactly rather than approximately, and that is stronger evidence that the two port defects we isolated account for the whole difference than the near-agreement we claimed before. It is also, plainly, a reproduction rather than a differentiating result, and we report it as one.

We first read the 0.41 points between our two runs of it as the per-task cap, since the two builds differed by nothing else we had declared. That reading was wrong, and its correction is the most useful thing in this section. An audit of the generated notebooks found the cap knob had never reached the solver at all. The build script rewrote a guard it expected to find as a local variable comparison, the source writes the same test inline, the pattern never matched, and so every submission from the first reproduction onward ran the same hardcoded 1800-second cap while its configuration banner printed whichever value we had set. The two runs were therefore the identical build at the identical cap, and their 0.41-point spread is run-to-run variance, which is worth about one task credit on 240 tasks. That is the noise floor for every hidden comparison in this work, and having it changes how the rest of the table should be read. One further observation follows for anyone reading the leaderboard rather than the method. A single task credit separates an entire sixty-entry cluster, so on this board the cheapest available gain is not a better model but a better use of the twelve hours.

The measurement run's own log then priced that claim. Of its 24821 task-seconds, 71 percent went to tasks that yielded nothing, no solve and no correct grid anywhere in the candidate pool, and the two longest tasks each burned to within sight of the 1800-second cap for nothing. One task in the forty trained normally and then decoded no output at all, a forfeited slot that scales to about four on the public 120 and plausibly eight on the hidden 240. Final training loss predicts these dead tasks weakly but usably at the extreme, cutting every task whose adapter finished above a loss of 0.05 reclaims 2330 seconds while touching no task the selector harvests, since the one such task with a correct grid in its pool was being lost at selection anyway. Three scheduler changes follow, none touching the method. A task whose search

returns nothing falls back to a plain greedy decode with its already-trained adapter, which can only fill otherwise-empty slots. A task whose training loss ends above the cut serves the cheap greedy answer and skips the expensive search. And the time both changes reclaim becomes margin for the per-task cap, the one knob already measured to buy a task credit. The next submission carries these as a scheduling variant of the unchanged pipeline.

The public board then handed us one more controlled measurement, and it is the largest of the three. A public notebook restoring this same pipeline is byte-equivalent to our restored variant in every module, the loader, launcher, decoder and solver all cancel in diff once its removed symbolic arm and comment noise are set aside, yet it scores **33.89** where our submission of the same code at the same 1200-second cap scored 30.56. Its kernel metadata names the only difference left. It mounts the original author's own dependency tree and model upload, where our builds mount a rebuilt wheel set whose pins predate the current execution image, and our own run logs had carried the tell throughout, a warning that compiled extensions were being skipped for a mismatched framework version. Identical code, weights, seeds and caps, submitted to the same hidden set from two software environments, differ by 3.33 points, about eight tasks. The dependency tree is a hyperparameter, and on this pipeline it is worth more than any method lever we measured this month. It also reframes the sixty-entry cluster at 30.97, ourselves included, as a population unknowingly running a numerically degraded environment. The engine lesson of Section 3.2 returns in its sharpest form, what carries the score is not only the method but the exact software it deploys on, and the next submission is rebased onto the 33.89 condition with the scheduler on top, so its delta attributes to the scheduler alone. That attribution did not survive either. The build in question scored extbf28.61, a regression, and the same audit that found the cap bug found it in this run too, so the comparison we had framed as identical code in two environments was in fact two different per-task caps as well. The dependency tree may still matter and we no longer have evidence that it does. What we have instead is a candidate mechanism that fits the whole table. On a 240-task rerun against a fixed twelve-hour wall, a looser per-task cap does not buy depth, it lets slow tasks consume the budget until the tail of the queue is dropped unstarted, and the public build we were chasing runs the tighter cap we had believed ourselves to be running. The next submission wires the knob for the first time and tests exactly that.

4.2 The engine speedup

The 13$\times$ decode improvement from changing engines (Section 3.2, Figure 2) is the single most important result for this hardware class. It turned an overnight, frequently-failing process into a sub-90-minute evaluation. On one consumer GPU, the inference engine dominated every other knob we touched (Figure 5).

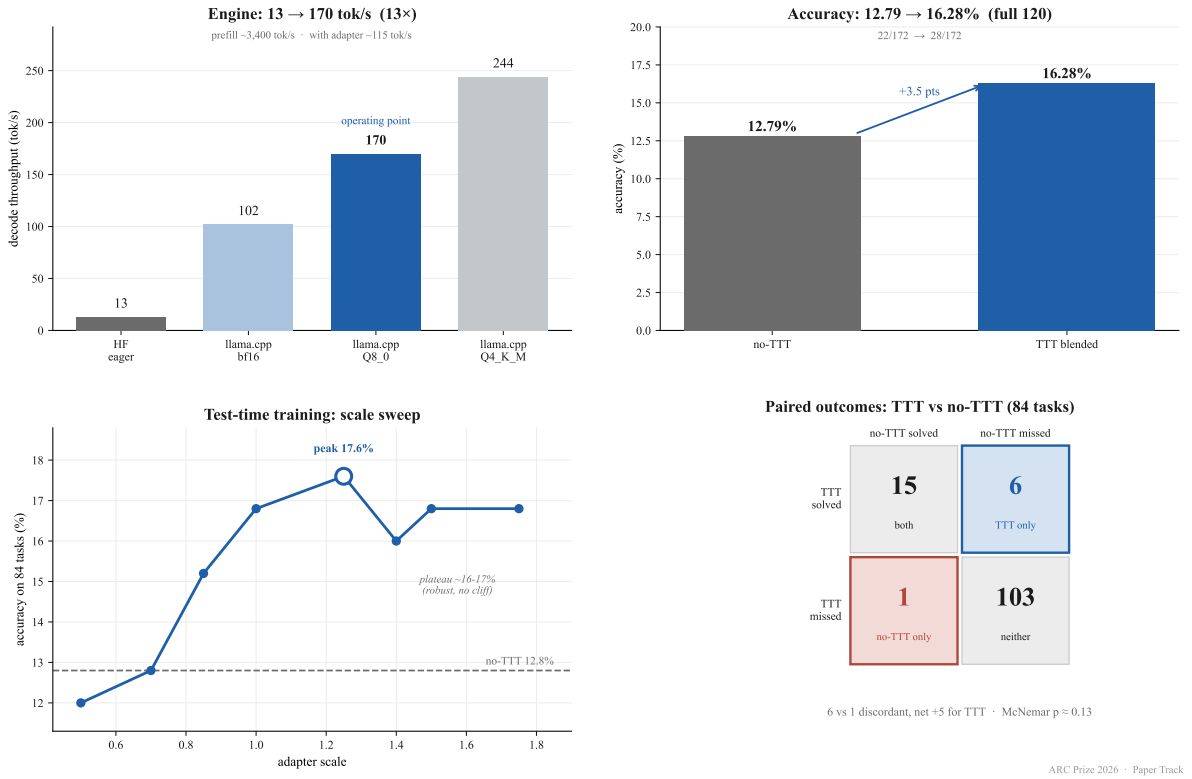


Figure 5: Engine choice, accuracy, and the adapter-scale curve. Accuracy peaks at scale 1.25 on the 84 trainable tasks, then declines, a real optimum rather than a monotone gain. All values are measured.

4.3 Test-time training

With generation fixed, we first swept the adapter configuration on 15 small-context tasks, scoring each against no-TTT on the same tasks (Table 3). The standard configuration overfits (its training loss reaches 0.000) and ties the baseline. The regularized configuration recovers a one-task lift. An over-aggressive reduction undertrains and ties again. That is coherent with an overfit diagnosis, though one task on 15 is only a hint.

Table 3: Adapter-configuration sweep on 15 small-context tasks (23 test outputs).

configuration	TTT	no-TTT
40 steps, rank 32, no dropout	3/23	3/23
20 steps, rank 16, dropout 0.1	**4/23**	3/23
8 steps, rank 8, dropout 0.05	3/23	3/23

We then validated the regularized configuration at scale, on the 84 evaluation tasks small enough to train on a 16 GB card, against no-TTT on the same tasks (Table 4). The lift holds and strengthens, adding $+5$ test outputs for a four-point gain ($\approx 31\%$ relative). The paired breakdown is clean. Of the tasks the

two methods disagree on, TTT is right on six that no-TTT misses and wrong on only one. With seven disagreements this is not individually significant (exact McNemar $p \approx 0.13$, Appendix B), but three measurements now agree in direction (sweep $+1$, scale $+5$, six-to-one discordant), and no-TTT scores 12.8% here against its 12.79% full baseline, so the subset is representative.

Table 4: Regularized TTT vs. no-TTT on the 84 trainable tasks (125 test outputs), scale 1.0.

method	correct	accuracy
regularized TTT	**21/125**	**16.8%**
no-TTT	16/125	12.8%

4.4 Ablations on scale, colour, candidates, filtering, and the big-grid wall

Five further levers, each measured on the 84 trainable tasks or the full evaluation (full per-scale numbers in Appendix B). The adapter *scale* matters. Accuracy peaks at 17.6% at scale 1.25, then declines (Figures 5, 6). Colour permutation lifts the no-TTT baseline to 14.4% but does *not* stack with the already colour-robust adapter. Forcing TTT onto the big-grid tasks does not help. We recorded the VRAM exclusion as a genuine ceiling rather than a missed opportunity, a conclusion Section 5.8 re-examines, since that early run trained under a 600-token context cap that admits a single demonstration on these tasks. The fourth lever is how many candidates the vote sees, and it cuts both ways. Sampling each of the eight D4 views four times raises the no-TTT score from 12.79% to 13.95% on the full evaluation, but like colour permutation it does not stack with the per-task adapter. Carried into a TTT submission those extra sampled candidates regressed the hidden score from 13.61% to 13.19%, and stacking multi-scale voting and colour permutation regressed it further to 12.78%. The adapter already concentrates the model on its single best answer, so more candidates help only the unsure no-TTT base and dilute the confident TTT one.

The fifth lever is a white-box plausibility filter adapted from a small-model recipe [7], which drops candidates that violate the task's colour palette or the train pairs' size pattern before the vote. On the public evaluation it is exactly neutral. The eight greedy D4 candidates are already plausible, so filtering changes no attempt on any of the 120 tasks, and where its inferred size rule misfires, on tasks whose train pairs coincidentally share one output size while the true rule varies, it empties the candidate list and the pipeline falls back to the unfiltered vote. Submitted as v5, it regressed the hidden score from 13.61% to 12.78%, a roughly two-task loss consistent with that size-rule tail landing on hidden tasks where the filter kept a plausible wrong candidate instead of emptying, though run-to-run TTT variance cannot be excluded from outside. The asymmetry is the lesson. A lever that measures neutral on the public set has no upside and only tail risk on the hidden one.

The eight greedy D4 views at a single scale are a local optimum that neither more votes nor vote filtering beats. The best blend, TTT at scale 1.25 where trainable and no-TTT elsewhere, reaches **16.28%** on the full 120-task evaluation, up from 12.79%. The honest summary is that naive per-task TTT *ties* a strong no-TTT AIRV baseline, while a regularized, scale-tuned TTT *beats* it by a consistent if modest margin.

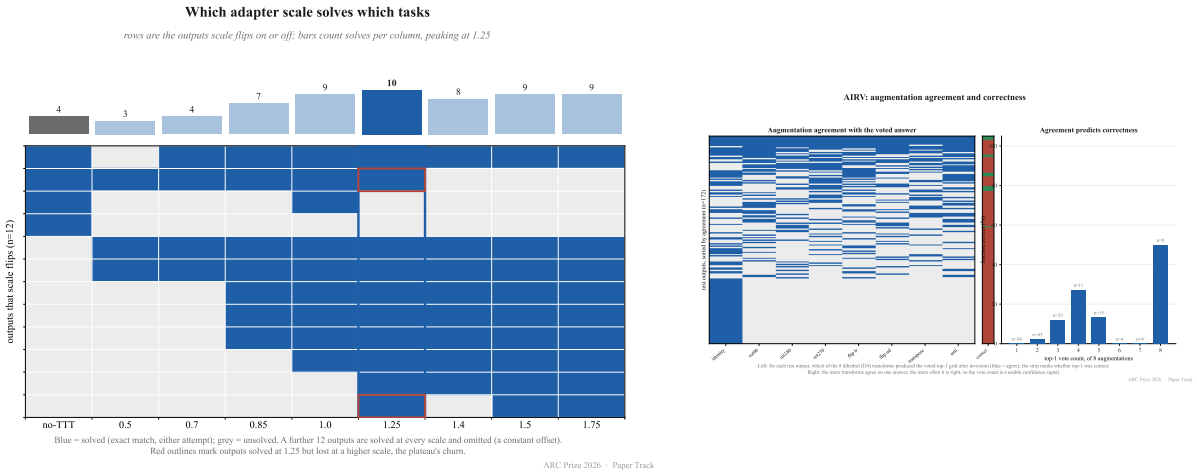


Figure 6: The left panel shows which tasks TTT solves at each adapter scale, with most gains concentrated at scale 1.25. The right panel shows AIRV vote agreement (Eq. (2)) predicting correctness, where full eight-way consensus is right far more often than split votes.

5 Discussion

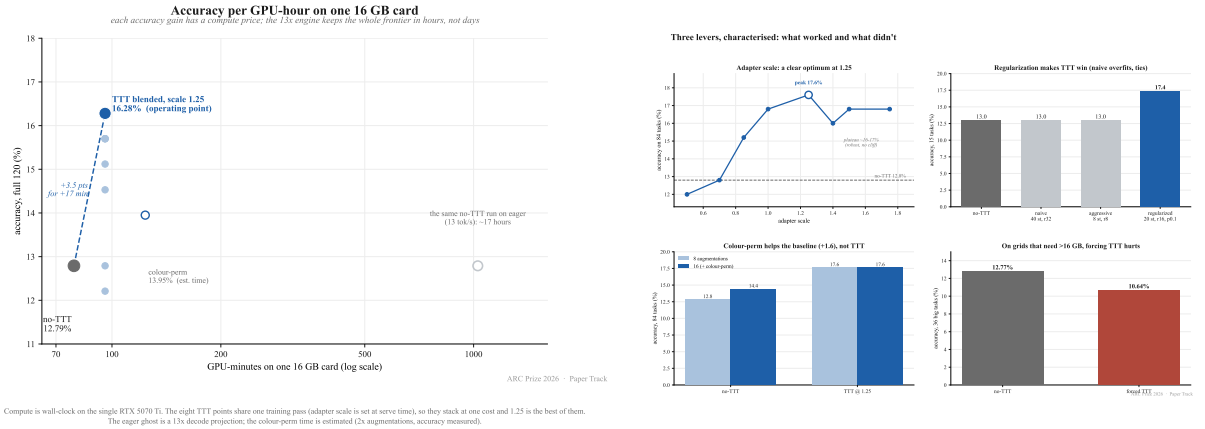
5.1 Four lessons that generalize

First, on a single GPU the inference engine is the dominant design choice. Quantization and sampling mattered far less than the 13× we recovered by moving decode off the eager path. Second, the real obstacle for TTT is the *serving bridge*. Training needs a gradient framework, fast generation needs an optimized engine, and per-task TTT produces a fresh adapter the engine must serve. We show this bridge is buildable on one card, and that once it exists the open question is not plumbing but *regularization*, because the small per-task training set overfits easily. Third, a 4B model is bimodal on these tasks. It solves or it misses with little partial credit, so the refinement family that defines the current frontier needs an external partial-credit signal to work at this scale. A method that only re-ranks the model's own outputs stalls (Section 5.3).

Fourth, public-eval gains must be re-validated in the exact deployed condition, because they do not transfer on their own. The level drops, the no-TTT system halves from 12.79% public to 6.53% hidden. More telling, the deltas reverse. The three candidate-side levers that lifted or tied the public evaluation, colour permutation with multi-scale voting, more-candidates sampling, and the plausibility filter, each regressed the hidden TTT score when submitted (Table 2). Two mechanisms compound. A lever validated on the no-TTT base meets a different system once the adapter has sharpened the vote, and a lever that measures neutral on one task distribution still carries tail risk on another. The operational rule this bought is that a lever counts as validated only in the exact submission condition, and a lever that measures neutral is never shipped.

5.2 Accuracy per GPU-hour

We argue the right framing for compute-poor ARC work is **accuracy per GPU-hour** (Figure 7), and the map of walls reported here (the decode ceiling, the VRAM limit on large-grid TTT, and the overfitting of naive adapters, all in Figure 7) is a first step toward it. The findings tell a practitioner with similar hardware exactly which problems to solve first. Fix the engine before touching anything else, budget VRAM by largest-train-pair length, and regularize the per-task adapter rather than training it harder.



Compute is wall-clock on the single RTX 5070 Ti. The eight TTT points share one training pass (adapter scale is set at serve time), so they stack at one cost and 1.25 is the best of them.
The eager ghost is a 13x decode projection; the colour-perm time is estimated (2x augmentations, accuracy measured).

Figure 7: The left panel shows accuracy per GPU-hour on one card, where the TTT blend at 16.28% sits on the Pareto front (compute is measured wall-clock, and any eager "ghost" points are 13× projections, labelled in the source). The right panel shows what worked and what did not. The engine swap and regularized TTT helped, while naive TTT, colour-perm stacking, and big-grid TTT did not.

5.3 Limitations and ongoing work

Our absolute accuracy is below NVARC's 24.0% winning result, by design. That score was set offline on four L4 GPUs, not a cluster, by a more developed recipe on the same Qwen3-4B base lineage, so the gap reflects recipe sophistication (synthetic data, ensembling, TTT scale) more than hardware. We study the one-card inference-plus-TTT recipe rather than maximize a system. The public-to-hidden gap is a finding, not a caveat (Section 5.1). Levels drop, the no-TTT system carries 12.79% public to 6.53% hidden while the TTT blend carries 16.28% to 13.61%, and deltas reverse, with every public-validated candidate-side lever regressing when submitted (Table 2). The public ARC-AGI-2 eval is the set every open recipe is tuned against, ours included, so the leaner the recipe the more of its public accuracy is public-specific. Part of the v1 drop may also be per-task failures in the scored rerun, which the later submissions instrument with a per-task log, and the TTT system's smaller gap is consistent with both readings. The 2025 analysis names verifier-guided selection and a refinement loop (check each candidate against the train pairs, then re-prompt on the discrepancy) as the central advance, so we built and tested it. It does not lift here. A passive form (weighting the augmentation vote by leave-one-out train fidelity) and an active form (verification-gated re-sampling on the tasks the base is unsure of) each match the plain no-TTT vote exactly on the eval slices we ran. The reasons are structural. The verifier is silent exactly where it is needed, since on a task the base misses it reproduces none of the held-out train pairs, so there is nothing

to select between or correct toward. Our 16-token model also cannot read a natural-language critique, so the canonical re-prompt-with-the-discrepancy step is not even expressible. We also probed whether a one-time continued LoRA fine-tune of the base on official ARC-AGI-2 training data and RE-ARC [6] generations yields a better static model. It did not. The fine-tuned base scores 11.63% no-TTT against the stock base's 12.79%, so we ship the stock base. The NVARC base is already well-adapted to this grid format, and continued fine-tuning on this public data did not transfer. That is a useful negative result for the compute-poor practitioner. More public data is not a free lever once the base is task-tuned.

Building on that null result, we tested an expert-iteration self-training loop. We solve the official training tasks with the current model, distil the base on a curriculum of its own verified-correct solutions, and iterate. Exact-match grids impose a sharp constraint. A verified-correct solution on a training task simply is its ground-truth output, so a naive self-training corpus coincides with the data the continued fine-tune already used. We hoped to escape this through the curriculum, distilling only the near-miss tasks the base reaches but does not solve robustly, but a 200-task harvest shows that set is nearly empty. The base self-solves 194 of 200 training tasks under some augmentation (97%) and 191 by the robust vote (95.5%), leaving only three near-miss tasks. The base is saturated on its own training distribution, so self-training has no signal to add. The training tasks split into already-solved (nothing to learn) and unsolved (no correct trace), with almost nothing between. This is a second, sharper negative result on weight refinement, and it explains the fine-tune failure directly. On one GPU, with a base already tuned to the grid format, neither continued fine-tuning nor self-training moves ARC-AGI-2, because the public training data carries no signal the base has not already absorbed.

These three null results, the fine-tune, self-training, and the refinement loop, share one cause. The base is bimodal on ARC. It solves a task robustly or misses it entirely, with almost no partial-understanding middle (of the training tasks it harvests, 95.5% are solved robustly and only 1.5% are near-misses). Every method that refines, re-ranks, or distils the model's own understanding therefore has nothing to grip on the tasks it misses, and per-task TTT, the one lever that helped, is the exception that proves the rule. It does not refine partial understanding. Instead it manufactures new capability for the task. The implication for compute-poor ARC is concrete. The next lever must supply an external, continuously-gradable signal that is valid even where the base is lost, a program checked against the train pairs being the cleanest, rather than another pass over the model's own grid outputs.

We built that arm and measured it. A 1.5B model trained to emit programs in an executable DSL, sampled and kept only when a program reproduces every train pair, solved zero of the 101 ARC-2 evaluation tasks our transductive system misses. It generalized weakly on the easier ARC-1, about one held-out task in nine, but on ARC-2 the DSL cannot express the new compositional concepts, and the tasks the base misses are also the largest grids where the small proposer exhausts the card before it can search. The external-signal lever is therefore real but bounded by model scale on one GPU, the same wall the refinement family hits from the other side. The lever exists but needs a larger model to grip, which is exactly the move the full contest hardware allows (Section 5.4).

Finally, the significance of the TTT lift is directional rather than conclusive (Appendix B). A full-eval paired run is the obvious next measurement.

5.4 Scaling to the full contest hardware

Everything above holds a single 16 GB card fixed, but the offline contest gives four L4 GPUs, 96 GB together, and that hardware is where the real climb happens. As of late July 2026 the live public leaderboard is led by one entry near 65.8%, with second place near 38.6% and a chasing pack in the mid thirties, and every one of those runs uses the same offline no-internet sandbox our system does. On the available evidence the leader is the same NVIDIA Kaggle Grandmasters lineage that won the 2025 offline track and whose grid model our system reproduces, and their stated recipe is to move all complex reasoning offline into a large synthetic-data pipeline and train a small model that runs fast at evaluation [3]. That framing matters for where the twenty-seven-point gap to second place actually lives. The contest's own 2025 technical report attributes the distance between the offline field and the higher application-track numbers to knowledge coverage and verifier-guided optimization rather than to any fundamental scientific breakthrough [3], which is to say the remaining lever is training-data scale and a working refinement or verification loop, not a decoding or selection trick of the kind we exhausted here. The higher numbers often quoted for ARC-AGI-2, a 54% Gemini refinement harness and frontier scores above 70%, come from a separate public track that allows internet and external compute, so they are not the bar we measure against [3]. The offline bar is the leaderboard above. The field runs test-time adaptation, heavy synthetic-data pretraining, and program search on the full 96 GB, and our best single-card score of 18.19% sits below it. Our faithful reproduction of the winning lineage's own pipeline, run on the four-L4 node it was built for, reached 30.97% and still sits far below the leader, which locates the gap in the base model and its training data rather than in the inference-time machinery we can run. None of our negative results argues against that climb, but none of them tests it either, so we state the scaling case as a falsifiable hypothesis for future work rather than as a claim. Every wall we mapped belongs to a 4B model on a single card, and the sharpest is bimodality. The model solves a task robustly or misses it entirely, which is the exact reason refinement and verifier-guided selection found nothing to grip. The hypothesis is that bimodality is a property of this model's scale and training rather than of the task. The falsifiable prediction is that a more capable base run in the 96 GB produces partially-correct candidates on tasks it does not solve outright, a measurable non-empty near-miss band where ours is 1.5%, and that this in turn reactivates the refinement and program-search levers that stalled here. Three of our measurements bound the hypothesis from below. A second small transductive model buys almost nothing, since our fine-tuned variant solves only two tasks the stock 4B misses. A union of our own two pipelines buys as little. The deep-TTT greedy vote and the restored DFS decoder share the same 4B and produce nested solved sets, the greedy arm adding exactly one task the decoder missed over the tasks both attempt, and that task already sat in the decoder's own candidate pool and was merely lost at selection, so the apparent diversity is a selection artifact rather than new capability. The Tiny Recursive Model family, the strongest small grid models available, scores far below our pipeline while needing more compute than one card gives it. That gap has not closed with time. The original TRM reports 8% on ARC-AGI-2, its 2026 denoising-recursion successor reaches only about 9.6% [3], and the winning team's own ensemble measured a TRM arm as adding nothing to a strong 4B, so any diversity that pays must come from capability rather than from a second small vote. One published result warns against reading the hypothesis as a bare parameter-count claim. A $\sim$200M encoder-decoder system with test-time training and white-box filtering reports 27% on the ARC-AGI-2 semi-private set, ahead of the 4B-based

2025 winner [7], so method quality can dominate size, and our own four failed elaborations of the 4B's candidate machinery (Table 2) sharpen the same point from below. We read the 16 GB result as the place where this small-model recipe runs out. The prediction above is the test of what replaces it, and we leave it as stated, a hypothesis with a measurement that would falsify it, rather than as a promise.

5.5 A routed dual-arm system

Section 5.3 argued that the next lever must supply an external, continuously-gradable signal that is valid where the base is lost, and Section 5.4 offered one escape, a larger non-bimodal base, as a falsifiable hypothesis. We are building a second escape that stays on the single-card efficiency budget rather than betting on scale, and it follows directly from two measurements already in this paper. The AIRV vote agreement predicts correctness (Figure 6), so the grid model reports for free which tasks it has solved and which it has no grip on. And the base is bimodal, so on a missed task there is nothing to refine, only something to replace.

The system routes on the first fact and fills the gap with a method that does not share the base's blind spot. High-consensus tasks are banked with the grid model's own answer, which is most of the current score at a fraction of the compute. Low-consensus tasks, where the transductive model scores near zero, are routed to an induction arm, a code model that writes plain-Python grid transforms which we execute against the train pairs. A program that reproduces every train pair is verified on observable data in a way no vote can be, so its output becomes the first attempt and the grid model's guess falls back to the second. The two arms are complementary by construction rather than by hope, induction expresses the precise compositional rules transduction misses.

The design is deliberately do-no-harm. A banked task is byte-identical to the transductive submission, and a program is trusted only when it reproduces every train pair, so the arm can only add coverage on tasks the base was already missing. This is the property the five candidate-side levers lacked. Each of them rewrote the vote on every task including the ones already solved, and each regressed (Table 2). The one loss channel, a program that fits every train pair yet is wrong on the test and displaces a correct second attempt, is measured on the local eval before any submission. The arm also inverts the engine economy of Section 3.2. Verifying a program by running it is cheap, and where several programs survive we rank their outputs under the grid model's per-view likelihood, which is prefill-priced and about thirty times cheaper than the decode that produced them, so the system generates less and checks more. The whole arm runs inside the idle two-thirds of the offline budget the transductive submission leaves unspent.

This is the same idea whose cheap form failed in Section 5.3, and it is built to answer why. That probe used a 1.5B model emitting a fixed ARC-1 domain-specific language and solved none of the 101 missed tasks. The arm here uses a 14B code model emitting unrestricted Python, which addresses both named causes at once, model capacity and an expressive target language.

Status. The first probe is a disjoint-coverage run on the single card. A 14B code model (Qwen2.5-Coder) samples sixteen programs per task at temperature 0.8, on exactly the 101 evaluation tasks the transductive system misses, and a program is kept only if it reproduces every train pair. It solved none of them. More telling than the zero, across roughly 1600 programs only one task saw a program reproduce even a single

train pair, so the model is not close anywhere. The programs are not the failure mode the arc-dsl proposer was. They are fluent and sensible. On a task whose output size changes with every example the model wrote a clean routine that extracts and sorts rows, which crashed because its fixed-shape hypothesis cannot produce a variable-size output, and on a task with a narrower output it wrote correct code for the wrong rule and returned the input width. The 1.5B DSL proposer failed because it could not express the concepts. This larger model expresses them freely and still cannot infer the rule from a few examples. The bottleneck is inference, not expression, and a general coder does not close it zero-shot on one card. Priming the coder with three verified worked-example solvers, one of them a variable-size crop aimed squarely at that failure, did not change the outcome. It solved none of the 101 and lifted the partial signal only from one task to two, with nothing reaching a near-miss. Two forms of a strong single-card coder, cold and primed, both come back empty, which is itself the result. The bottleneck is rule inference on the hardest tasks, and neither sampling more programs nor showing worked examples closes it at 14B on one card. A same-size reasoning model, which works through the transformation at length before it writes code, does not change the verdict either. On a bounded run over twelve of the missed tasks it solved none and reproduced no train pair, at roughly four minutes of reasoning per sample. Three model classes, a small restricted proposer, a strong instruct coder cold and primed, and a reasoning model, all fail to induct these hardest tasks on a single card, and the near-miss band stays empty throughout. We read this as a decisive single-card result rather than a tuning gap. The router is sound, vote agreement does report which tasks the grid model has solved, but the arm it would route to cannot grip the tasks the grid model misses without more compute than one card provides. That is the same wall Section 5.4 predicts, now measured from the induction side, and it is where the second arm's value waits on the full contest hardware.

5.6 A stronger public decoder, and its coupling to training

The clearest evidence that the gap in Section 4 is method rather than model arrived from outside the project. A public notebook ports the 2024 ARC-Prize-winning pipeline, the ARChitects', onto our exact base and reports a score in the low thirties on ARC-AGI-2, more than double our operating point on the same weights. The port's lineage deserves naming. The pipeline is published by Ivan Sorokin, the author of the NVARC base model itself, as the notebook `arc2_qwen3_unsloth_flash_lora_batch4_queue`, and the variant we study is its community re-target to the 2026 contest. Everything we report about this system is reproduction and measurement of that public work, credited to its authors, and our contribution is confined to the analysis around it. It bundles two changes against our recipe. The per-task training is an order of magnitude heavier, a rank-256 adapter that also trains the embeddings and the output head at an eight-thousand-token context, where ours is rank sixteen at two thousand. And the decoder is not our greedy vote. In its place it runs a depth-first search over every completion whose running probability stays above a fixed floor, then selects among the survivors by the product of their likelihoods across fresh augmentations rather than by a majority.

We had named this decoder as the untried lever ourselves and never built it, so the first question is whether it is a separable part we could capture on one 16 GB card without the heavy training it ships with. We test it directly. Holding the adapter fixed at our rig-feasible configuration, we decode each task twice on the identical trained model, once with our greedy vote and once with the search-and-select, and score both against the solution. Only the decoder moves.

On our light adapter the search loses, and it loses for a reason that names the coupling. Over eighteen of the trainable tasks the greedy vote scored the equivalent of 5.17 solved tasks to the search's 3.33, 28.7 percent against 18.5. The mechanism is visible in the candidate counts. On task after task the search returns nothing at all, because no whole-grid completion clears the probability floor. A grid runs to about nine hundred tokens, and a little uncertainty at each cell compounds across all of them, so an adapter that is not sharply peaked pushes the probability of even its best completion under the floor. That floor is calibrated for a model trained hard enough to stay almost certain across the whole grid. Ours is not, so the search starves and forfeits tasks the greedy path still lands. The decoder is not a model-agnostic component we can lift. It is tuned to the sharp adapter the heavy training produces, and the two are one system rather than two levers. This is also why we could never have reached the field's level by swapping decoders on the pipeline we already had.

Status. Resolved, and the coupling did not save the decoder. Over thirty trainable tasks the rank-128 adapter, with the embeddings and output head trained as far toward their configuration as a 16 GB card holds, gives the greedy vote **24.4%** against the search's **13.3%**. The search fell further behind than it was at rank 32, not closer, because the sharper adapter still starves the probability floor on most tasks while the heavier training only helped the vote. On every configuration a 16 GB card can run, rank 32 and rank 128, the search loses to the vote. We read this as the decoder being inseparable from the full rank-256, eight-thousand-token training it ships with, which one card can hold but cannot serve inside the contest budget (Section 4), so the reach to the field's low thirties stays a claim to be settled on the contest hardware rather than a lever we could capture here. The lift we did capture, from 13.61 to 18.19, came from the training depth alone. A first hidden-set submission of the reproduction itself, run faithfully with the community port's bolted-on symbolic arm still active, scored 16.81%. Two facts inside that number matter more than its size. The public-to-hidden transfer is flat, 16.2% on the evaluation slice against 16.81% hidden, where every candidate-side lever we built ourselves lost points crossing that gap. And the symbolic arm is measured poison, on the evaluation slice it preempted twelve of thirty-nine tasks with its own answer and got none of them right, so the faithful reproduction understates the pipeline. The arm is stripped and the corrected variant is measured next. That measurement is in, and it also surfaced a second inherited defect, the community port had split the decoder's four-view batches into pairs, halving search throughput. With the symbolic arm removed and the original batching restored, the same evaluation slice rises from 16.2% to **27.2%**, no tasks preempted, no timeouts, and half again more correct answers present in the search beams. Eleven points of the gap between the reproduction and its source's claim were the port's own mutations. Submitted to the hidden set, the restored variant scored **30.56%**, against 16.81% for the same pipeline as ported and 18.19% for our own best single-card configuration. Three things follow. The transfer property held a third time, 27.2% on the evaluation slice against 30.56% hidden, so this pipeline carries its public number across the gap that every candidate-side lever of ours fell through. The original author's claimed tier is credible, since a faithful restoration lands within striking distance of it while the ported form did not. And the result is a reproduction, not a contribution of ours. What this work adds around it is the measurement, the isolation of the two port defects, the paired evaluation that priced each one, and the placement of the restored number against a single-card baseline so the hardware gap becomes visible.

5.7 The selection leak, measured on the candidate pool

A search-and-select decoder is only as good as its selector, and the restored pipeline of Section 5.6 throws away correct answers it has already found. To measure how many, we ran a build identical to the submitted one except for a gated dump of the entire candidate pool. It records, for every test output, each distinct candidate grid together with the number of augmented views that produced it, the beam likelihood of each view, and the per-augmentation likelihoods the selector consumes. Over forty evaluation tasks the run produced 60 test outputs across 39 tasks, one of the forty yielding no decoded output at all, holding 432 distinct candidate grids drawn from 847 decoded subkeys, of which 152 reproduce the ground truth exactly. Scoring follows the harness, where each task contributes one credit divided among its test outputs.

The dump is a replica of the deployed decision rather than a model of it. Reimplementing the shipped selector against the file returns 9.833 credits of 39, equal to the run's own scored output, so a candidate rule tested offline here is tested against the real thing at no GPU cost.

Table 5: Selection against the pool ceiling on the 40-task measurement run (60 test outputs, 39 tasks).

selection rule	outputs solved	credits
published vote, deployed	15	9.833
confidence-weighted vote	15	9.833
augmentation likelihood alone	16	10.333
correct grid present in the pool	**19**	**12.333**

The leak is large. Nineteen of the 60 outputs hold a correct grid somewhere in their pool and the deployed rule converts fifteen, 9.833 credits against a ceiling of 12.333, so 2.5 credits or 6.4 points of this slice sit inside candidates the system generated and then ranked away. The published rule and the confidence-weighted alternative select identically on all 60 outputs, neither gaining nor losing one, which is a third independent reading that the two are near-equivalent once deployed.

We then asked whether any reweighting of the statistics already in the pool closes that gap. Sixteen rules were scored against the dump, spanning the vote count alone, the beam likelihood alone, the augmentation likelihood alone, the deployed count-minus-penalty form at five weightings, a best-view variant taking the minimum augmentation penalty rather than the mean, and a length-normalised family motivated by summed log-likelihoods penalising physically larger grids. Exactly one rule gains. Ranking by augmentation likelihood alone, discarding the vote count entirely, recovers one output and loses none, for sixteen solved and 10.333 credits. The length normalisation is inert, since only 11 of the 60 outputs contain candidates that differ in size, and the normalised and unnormalised forms return identical solved sets.

Table 6: The four outputs whose pool holds a correct grid the deployed selector ranks outside its two attempts. Views is the number of augmented decodes producing that grid, penalty its mean augmentation negative log-likelihood, where lower is better.

test output	pool size	correct grid (views / penalty)	selected grid (views / penalty)
`142ca369_0`	10	3 / 18.64	4 / 10.57
`3dc255db_0`	13	2 / 13.57	1 / 7.36
`3a25b0d8_0`	7	1 / 5.96	7 / 4.69
`4c7dc4dd_1`	11	1 / 12.90	3 / 8.79

That gain does not survive inspection, and the reason is in the four outputs the deployed rule misses (Table 6). In every one the correct grid carries a heavier augmentation penalty than the grid the selector chose, so the signal does not merely fail to surface these answers, it actively orders them below the wrong one. On one output the correct grid was decoded more often than the grid that beat it and still lost, on the penalty term alone. The single recovered output is a correct grid moving from third place to second, which slides it into the second attempt slot while still losing outright. Two of the four are reachable by some rule in the family, each time at the price of losing outputs the deployed rule already holds, and the other two are reached by nothing we tried. On three of the nineteen recoverable outputs the correct grid was decoded exactly once in the whole pool, so no vote-based rule can reach them in principle.

The exercise also carries a warning about the slice it ran on. An earlier reading of these same forty tasks found three discordant outputs favouring the confidence-weighted rule over the published one, with a clean mechanism behind it, so we shipped the switch, and a 120-task run then reversed it. This sweep points the opposite way on the same tasks, that discarding the vote count beats keeping it, on the evidence of one output in sixty and as the best of sixteen rules tried. Two contradictory prescriptions of equal statistical weight, from one evaluation slice. We therefore record the count-free rule as measured and leave the deployed selector alone.

What we carry forward is the negative, and it is sharper than the headroom number by itself. The selection leak is worth roughly six points on this slice, and it is not recoverable by rescoring what the pipeline already computes, because every statistic in the pool ranks the discarded correct grid below the chosen one. Closing it needs a signal the pool does not contain, consistency against the task's own train pairs, which is the verifier direction Section 5.3 finds silent exactly where it is needed. The contribution of this measurement is to eliminate the cheaper alternative. Reweighting is exhausted, and the instrument for any future attempt is the pool dump itself, which makes selection rules testable offline without further contest compute.

The signal that paragraph asked for has since been built, and it fails too. It is worth reporting because the construction avoids the objection that sank our earlier verifier, which scored an adapter rather than a candidate and so could not choose between two candidates the same adapter produced. Here a candidate is scored as evidence. An output proposed for the test input is either a true instance of the task's rule or it is not, so appending the pair of test input and candidate to the task's demonstrations either hands the model a fourth true example or hands it a contradiction. We measure the negative log-likelihood of a held-out real demonstration's output twice, once from the remaining demonstrations alone and once with

the candidate injected, and average the difference across held-out choices. A negative delta means the candidate behaved like a true example of the rule. Scoring a difference rather than a likelihood cancels per-task and per-fold difficulty, and the whole thing costs forward passes rather than decodes, so it is affordable over every candidate inside the contest budget.

A cheap gate ran first, over all 172 evaluation test outputs, scoring the true grid against four distractors, the test input unchanged, the true grid with two colours swapped, the true grid transposed, and another task's answer. The true grid ranked first on 133 of them against a chance rate of one in five, which read alone looks like a working signal. The margins say otherwise. The median winning margin is 0.0043 nats and the median losing margin is 0.0031, so the two distributions sit on top of each other, and 58 percent of the wins are by less than 0.005. The shape of the failures matters more than the rate. Another task's answer loses 171 times out of 172, so the score reliably rejects a grid unrelated to the rule, while transpose beats the truth 17 times, colour swap 11 and the unchanged input 10. Those three preserve the structure of the answer and change its content, which is precisely what a decoder's near-misses are.

Table 7: Round-trip candidate scoring against the deployed selector, on the same 40-task pool as Table 5. Credits are out of 39.

selection rule	outputs solved	credits
published vote, deployed	15	9.833
round trip alone	7	5.500
published vote, round trip as tiebreak	15	9.833
correct grid present in the pool	**19**	**12.333**

Against the real pool it is worse than the rule it was meant to improve (Table 7). Ranking by the round trip alone solves 7 of the 60 outputs where the deployed rule solves 15, losing eight and gaining none, and applied as a tiebreak it changes nothing at all, because the deployed rule produces no exact ties to break. On the four outputs the leak consists of, the correct grid ranks seventh of ten, seventh of seven, tenth of thirteen and fourth of eleven. A merely noisy score would scatter those near the middle. Landing in the bottom half on three of four says it is measuring something real and something other than correctness.

That something is grid complexity. A simple, repetitive, low-entropy candidate is cheap to append and perturbs the model very little, so its delta sits near zero, while a complex correct answer perturbs more. The score therefore rewards candidates that are easy to encode over candidates that are right, which is also why the unchanged test input, usually the structurally cheapest grid on offer, beat the truth ten times in the gate.

One qualification is owed. The scoring ran against the unadapted base model, because the per-task adapters that produced this pool were discarded with the contest container, and a sharper adapted model would react more strongly to a contradictory demonstration. We report the number as a floor for that reason. It is not a floor we expect to lift far, because the failure is systematic rather than weak, and a complexity bias does not dissolve under better adaptation.

Two further attacks followed, on the attempt slots and on the grids themselves, and both close the same

way. The first observed that our two attempts come off a single ranking, so when the top two candidates are near-duplicates the second slot restates the first, and tested spending attempt two on the highest-ranked candidate at least k cells away from attempt one. At every k tried the rule loses one output and gains none, because the geometry runs opposite to its premise. On `4c3d4a41_1` the correct grid is rank two, one cell from rank one, so any diversity requirement skips it, and on `3dc255db_0` the correct grid ranks sixth while sitting two cells from the chosen wrong answer. When this pipeline misses, its miss is a near-duplicate of the truth, and the distant candidates a diversity rule promotes are noise.

That anatomy suggested the opposite mechanism, fusing the near-duplicates instead of avoiding them. A per-cell weighted vote across same-shape candidates can emit a grid that appears in no candidate, which grid-level voting cannot do by construction. It emits nineteen novel grids on this pool and not one of them is correct. Count-weighted fusion in the second attempt slot is exactly neutral, and every other weighting regresses. The failure case is the informative part. On `3dc255db_0` thirteen same-shape candidates fuse to a grid two cells from the truth, and they are the same two cells the selected answer gets wrong. The candidates' errors are correlated, so no vote among them can cancel what they all agree on.

We stop here, and the stopping is the finding. The leak is the largest measured gap in this system, 6.4 points on this slice and roughly fifteen task credits at contest scale, and it has now turned back four attacks with unrelated mechanisms, sixteen reweightings of the pool's own statistics, an out-of-pool consistency score that ranks the answers backwards, a diversity-forced second attempt, and cell-level fusion. The correlated errors explain all four at once. The model does not hold a noisy version of the right rule that clever aggregation could denoise, it holds the wrong rule, and every candidate it produces expresses that same wrong rule. This is the bimodality of Section 5.4 surfacing at cell level, and it relocates the leak from a selection problem to a training problem. The information needed to pick the correct grid is absent from the pool and from the model that produced it, so any repair has to change what the model believes, not how its outputs are ranked.

5.8 Revisiting the big-grid wall with the host's memory

The journey of one cap tells this subsection's story. Thirty-six of the 120 evaluation tasks have a train pair larger than a thousand tokens, and their per-task training runs out of memory on 16 GB, so from the first TTT submission onward they were excluded and served without adaptation. We tested that exclusion once, early, by forcing TTT onto them with the training context capped at 600 tokens, the most that then fit. It scored 10.6% against 12.8% without, and we recorded the loss as "the VRAM exclusion is a genuine ceiling, not a missed opportunity." That sentence survived every draft since. Rereading the measurement now, it does not support the sentence. A single train pair on these tasks is at least 1010 tokens, so a 600-token cap admits exactly one demonstration into the training context, and the adapter learned each step from a single example where the small tasks see several. What we measured was truncation, not the ceiling.

The second look came from turning away from the card. The rig carries 64 GB of system memory that sits idle while the 16 GB of VRAM starves, and during TTT the base model is frozen, only the small adapter trains, so the 7.5 GB of frozen weights never need to live on the GPU at all. Streaming them from host memory through a device map, with a 6 GiB resident-weight budget on the card, frees the VRAM for what

actually grows with grid size, the activations, and those stay small here because the 16-token vocabulary keeps the output layer tiny even at long sequence lengths. Under that arrangement the training context rises from the old 600-token cap to 8000 tokens, above the 7471-token worst case across all thirty-six tasks, which is to say literally full context, the same leave-one-out signal the small tasks have always had. The cost is streaming time, the frozen weights cross the PCIe bus every step, which is unaffordable inside a contest budget and nearly free on the rig.

Status. The staged probe has run its first two gates and the ceiling sentence is already refuted in its strong form. The smoke stage trained the single largest excluded task end to end, 14.6 GB peak on the 16 GB card with the base streaming from host memory, every mechanical check passing. The six-task stratified panel then produced the first solve this excluded set has ever yielded, task 8e5c0c38 moves from zero of two test outputs without adaptation to one of two with its full-context adapter, and no panel task got worse. So at least one of the thirty-six is winnable, and what was recorded as a genuine ceiling was, at least in part, the truncation artifact Section 4.4 now flags. Training losses tell the mechanism, adapters on these tasks now converge to near zero where the 600-token cap never could. One operational wrinkle surfaced, on the longest-context tasks Windows silently overcommits GPU memory into host RAM rather than failing, which crawls training, and the affected adapters were retracted and retrained at a lower resident-weight budget. The full thirty-six run is complete and the count is decisive. Across the excluded set's 47 test outputs, the unadapted system solves 4 in this paired harness and the full-context adapters solve **8**, five tasks flipping to solved against one flipping away, that one a displacement the two-attempt format removes in deployment by carrying both arms. Four of the largest tasks remain honestly flagged undertrained where even the reduced-budget training exceeded its time cap, so the 8 is a floor with headroom. The doubled count settles the question this subsection opened. What we recorded as a genuine ceiling was in substantial part a truncation artifact, and the excluded third of the evaluation is partially winnable on this card with the host's memory carrying the frozen weights.

One postscript, because it may make the streaming unnecessary. The activation growth that forced the frozen weights off the card in the first place was measured under an attention kernel the framework had chosen by default, and forcing the cuDNN backend instead trains rank 256 at eight thousand tokens fully resident in 14.86 GB (Section 4). Our big-grid configuration is rank 32, well below that, so these tasks would plausibly train resident and pay no streaming cost at all, which would move the arrangement from unaffordable inside a contest budget to ordinary. We have not measured that cell, so we state it as the obvious next probe rather than as a result, and we leave the streaming numbers above as they were measured.

6 Conclusion

The 2025 winning recipe survives a single 16 GB consumer GPU, but only after the inference engine is fixed. That one change is worth more than every other knob, and it is what makes per-task TTT tractable through an adapter-serving bridge. The honest result is a 12.79% no-TTT baseline and a 16.28% regularized-TTT blend on the public evaluation, 13.61% and then 18.19% on the hidden leaderboard as the per-task training deepened, with the gains bounded by adapter overfitting and by a VRAM wall on large grids that later proved to be partly a context-truncation artifact rather than a true limit. Against that

single-card ceiling we set one measurement from above. A faithful reproduction of a public pipeline that one 16 GB card can hold but not serve in budget, run on the contest's own four-L4 node and repaired of two defects its community port had introduced, scores 30.97% on the same hidden set. That result is not ours and not single-card, and its value here is that it prices the constraint at roughly twelve points instead of leaving it an open question. The sharpest general lesson is that public-eval gains must be re-validated in the deployed condition, since every public-validated candidate lever we submitted regressed on the hidden set. Those bounds belong to one small card, not to the task. The contest gives four L4 GPUs and 96 GB, where the offline frontier now leads near 66%, and the verifier, the TTT bridge, and the selection rule we built are the pieces we would carry onto that hardware, with the scaling case stated as a falsifiable hypothesis rather than a promise (Section 5.4). The 16 GB card is where the climb starts, not where it ends, and the number beneath it keeps moving. The full system is open source, the GGUF conversion recipe for the custom-vocabulary model, the AIRV pipeline, the per-task TTT bridge and sweep harness, and a development log recording every measurement and dead end behind these numbers.

A Reproducibility

Environment. A single NVIDIA RTX 5070 Ti (16 GB, Blackwell, $\approx$ 896 GB/s), native Windows 11. For inference, a prebuilt llama.cpp CUDA 12.4 `llama-server`, run on Blackwell through PTX JIT. For training, PyTorch with a LoRA via PEFT, bf16 base, gradient checkpointing. The two frameworks are never co-resident in VRAM (Section 3.4).

The two GGUF converter patches. The NVARC model has a 16-token *WordLevel* tokenizer, which trips the stock converter twice. (1) *Unknown tokenizer hash:* the loader does not recognize the tokenizer's hash and refuses to guess a pre-tokenizer. Its pre-tokenizer is in fact the `qwen2` regex, so we map the hash to `qwen2` explicitly. (2) *Empty merge list:* a WordLevel tokenizer has no byte-pair merges, but the loader requires a non-empty list, so we add two throwaway merges among the role tokens, which never fire on grid text. Both patches are in the repository's conversion script and are applied automatically by the Kaggle prep notebook.

Serving flags. `llama-server` is launched with full GPU offload (`-ngl 99`), a 16,384-token context, a single slot (`-np 1`), and the prompt cache disabled (`-no-cache-prompt`). Its own logs are redirected to a file so they never flood the notebook. Decoding is greedy (temperature 0) with `n_predict` sized to the target grid.

Hyperparameters. AIRV uses the eight D4 augmentations and votes to two attempts. Per-task TTT (regularized): rank 16 to 32, dropout 0.05 to 0.1, 8 to 20 steps, AdamW, a fresh random augmentation per step, leave-one-out target, context capped at $\approx$ 1,600 tokens, applied at adapter scale 1.25 at inference. Exact values per experiment are in the development log.

B Extended results

Per-scale accuracy (84 trainable tasks). Table 8 gives the full adapter-scale sweep behind Figures 5 and 6. Accuracy rises to a peak at scale 1.25 and then declines, the signature of an adapter applied too strongly.

Table 8: No-TTT vs. TTT accuracy on the 84 trainable tasks as a function of adapter scale.

adapter scale	no-TTT	0.50	0.70	0.85	1.00	**1.25**	1.40	1.50	1.75
accuracy (%)	12.8	12.0	12.8	15.2	16.8	**17.6**	16.0	16.8	16.8

Paired significance. On the 84 trainable tasks, TTT and no-TTT disagree on seven test outputs. TTT is correct on six that no-TTT misses and incorrect on one that no-TTT gets. An exact two-sided McNemar test on a $6/1$ split gives $p \approx 0.13$, so the lift is not individually significant. We report it as directional, corroborated by the configuration sweep ($+1$) and the representativeness check (no-TTT scores 12.8% on the subset versus 12.79% on the full set).

Colour permutation. Adding a random colour permutation to the augmentation set lifts the no-TTT baseline from 12.8% to 14.4% on the 84 tasks, but provides no further gain on top of the per-task adapter, which has already learned the task's colour mapping, so the two do not stack.

Deep test-time training and the training context. The configuration sweep in Table 3 conflated two variables. Its deeper cell ran 40 steps at rank 32 with no dropout and its shallow cell 20 steps at rank 16 with dropout, so more steps was only ever measured with the regularizer removed. Table 9 separates them on all 84 trainable tasks, trained at the 2000-token context the submitted system uses. Depth alone does nothing. Sixty steps at rank 16 with dropout ties the twenty-step baseline at 24 of 125. Adding capacity while keeping the regularizer on is what moves the number. Sixty steps at rank 32 reaches 27 of 125, the best trainable-task result we have measured, and it prefers the same 1.25 inference scale. The honest caveat is variance. Two seeds of the baseline configuration score 24 and 22, so a single output sits inside the noise, and the deep result rests on one seed. A second seed is the confirmation and its verdict is in the status below. The same sweep isolates the training context as a real knob. The baseline trained at 1600 tokens scores 16.0% against 19.2% at 2000, a three-point gap wider than the seed noise. This reconciles a measurement detail. The trainable-task figure in Section 4.3 and the blend it feeds were measured at the 1600-token context of the sweep harness, while every submission from v2 on trains at 2000, so those reported numbers describe a lighter configuration than the leaderboard one and are a conservative floor for it.

Table 9: Deep-regularized TTT sweep on the 84 trainable tasks (125 test outputs). Dropout is 0.1 and the training context is 2000 tokens except where noted.

configuration	steps	rank	scale	context	correct/125 (%)
baseline (v2.1 config)	20	16	1.25	2000	24 (19.2)
baseline, seed 1	20	16	1.25	2000	22 (17.6)
deep	60	16	1.25	2000	24 (19.2)
deep	60	16	1.00	2000	23 (18.4)
deep + rank	60	32	1.25	2000	**27 (21.6)**
deep + rank	60	32	1.00	2000	25 (20.0)
deep + rank	60	64	1.25	2000	28 (22.4)
deep + rank	60	128	1.25	2000	24 (19.2)
baseline, short context	20	16	1.25	1600	20 (16.0)

Status. Confirmed. A second seed of the rank-32 deep configuration scores 26 of 125 and the first scored 27, while the two baseline seeds score 24 and 22, so the two configurations separate with no overlap and the roughly three-output lift is a real effect rather than a seed draw. This is the first configuration we have measured that beats the shipped operating point on the mechanism that works, per-task training itself, rather than on the candidate-and-voting side where five submissions regressed. It cleared it. Shipped as v6.1 with the deep-regularized adapter trained on all 232 tasks, it scored **18.19%** on the hidden set, a 4.58-point gain over v2.1's 13.61% and the first submission to beat the shipped operating point (Table 2). The transfer held because this lever improves the adapter rather than diluting the vote, so the local sweep lift predicted a real hidden gain rather than the public-to-hidden reversal that sank the candidate-side levers. An intermediate v6 scored 12.36% as the same recipe starved by a disk-exhaustion bug that left 73 tasks without an adapter, and fixing the scratch path recovered the lift. The rank dimension is now swept to its ceiling. At the same 60 steps, rank 64 scores 28 of 125 against rank 32's 27, one output and inside seed noise at double the adapter cost, and rank 128 falls to 24, an inverted U whose mechanism is the per-task adapter overfitting its tiny few-shot dataset once capacity outruns the signal. The shipped configuration sits at the peak of its own curve, so the depth-and-rank lever is closed at 18.19% and further climb on this pipeline must come from elsewhere.

That closure had a built artifact attached to it. A rank-128 submission had already been assembled from the v6 recipe on an early partial reading that higher rank helped the vote, and it was **retired without ever being submitted** once the full sweep placed rank 128 joint-last, below the rank 32 it forked from. Rank 64 leads rank 32 by a single output out of 125, which is inside seed noise at twice the adapter cost, so no rank on the curve justifies a submission. The build is archived in the repository rather than deleted, because the episode is one of the paper's arguments made prospectively rather than in hindsight. Every candidate-side lever we submitted on a partial reading regressed on the hidden set. This one failed its validation first and cost a rig afternoon instead of a submission slot.

C Engineering log of walls and dead ends

We record the failures explicitly, because on this hardware they are most of the work.

- **Eager decode is overhead-bound, and the usual fix is unavailable.** Hugging Face eager decode runs at ≈ 13 tok/s, about 10% of the bandwidth roofline. `torch.compile` would close the gap, but it needs Triton, which has no native-Windows backend. The resolution was to change engines, not to optimize the eager path.
- **The server prompt cache destabilizes unique-prompt workloads.** Enabled by default, it reserves ≈ 680 MiB per unique prompt. Every ARC prompt is unique, so it never reuses and instead destabilizes sustained generation. Disabling it and running a single supervised stream fixed the stalls.
- **4-bit QLoRA is broken on Blackwell + Windows.** bitsandbytes fails to load on this platform, so the planned 4-bit continued fine-tune was infeasible. Because per-task TTT already runs in bf16 inside 16 GB, we use a bf16 LoRA throughout and drop the 4-bit path entirely.
- **PyTorch decode can turn quadratic after a training step.** Generation in the gradient framework is not only slow but intermittently degrades after an optimizer step. This is precisely why the serving bridge exists, since we never generate in PyTorch in the scored pipeline.
- **Hugging Face eager stalls offline validation, the same engine lesson again.** The rig validation scripts that drove generation through eager had no per-task timeout, so one big-grid task could hang a whole multi-hour run with no way to resume. Moving validation onto the same `llama-server` path used for scoring, with a hard per-request cap, a heartbeat file, and per-task checkpoints, removed the stalls and made the runs resumable. The engine was the bottleneck for validation just as for inference.
- **The VRAM wall on large grids was a default, twice over.** The largest-train-pair token length, not the test grid size, is what grows the training sequence. We first recorded the resulting exclusion as a hard limit, then found that forcing TTT onto those tasks had only ever been tested under a six-hundred-token truncation that admits one demonstration, and later that the framework was dispatching attention to a quadratic kernel because no flash or memory-efficient implementation exists for this GPU in this build. Forcing the cuDNN backend trains rank 256 at eight thousand tokens in 14.86 GB. The general lesson is to distrust a wall that was never probed one variable at a time.
- **Naive TTT ties the baseline.** The standard adapter configuration drives training loss to 0.000 and generalizes no better than no-TTT. Only regularization (fewer steps, lower rank, dropout) recovers a lift.

D Prompt and serialization format

The model's 16-token vocabulary is given in Table 10. A grid is serialized row by row, one token per cell (the token id equals the colour), rows separated by the newline token. Prompts are assembled at the id level as a ChatML few-shot sequence of train pairs followed by the test input, ending at the `ASSISTANT`

turn for the model to complete. Because the role and special tokens have no textual spelling, the whole sequence is built from ids directly, never by tokenizing a string, which is what guarantees parity between training and inference.

Table 10: The 16-token grid vocabulary.

token ids	meaning
0 to 9	the ten ARC grid colours (id = colour)
10	newline / row separator
11	`USER` role marker
12	`ASSISTANT` role marker
13	`PAD`
14	`IM_START`
15	`IM_END`

E Data and model provenance

The base model is Qwen3-4B [5] (Apache-2.0), supervised-fine-tuned on ARC grids by NVARC [4]. Any derived weights we redistribute are released under Apache-2.0 with the required `NOTICE`. Evaluation and training tasks are the official ARC-AGI-2 corpus [2] (Apache-2.0). The fine-tune probe additionally uses RE-ARC [6] (MIT). Our own code and weights are released under MIT-0. We never train on the evaluation tasks. Full license texts and attribution are in the repository's `THIRD_PARTY_LICENSES.md` and `NOTICE`.

BibTeX.

```
@article{chollet2019measure,
  title={On the Measure of Intelligence},
  author={Chollet, Francois}, journal={arXiv:1911.01547}, year={2019}}
@techreport{arcagi2,
  title={ARC-AGI-2 Technical Report},
  institution={ARC Prize Foundation}, year={2025}}
@techreport{arcprize2025,
  title={ARC Prize 2025: Technical Report},
  institution={ARC Prize Foundation}, note={arXiv:2601.10904}, year={2026}}
@misc{qwen3, title={Qwen3 Technical Report}, author={Qwen Team}, year={2025}}
@misc{nvarc2025,
  title={NVARC: ARC Prize 2025 (ARC-AGI-2) 1st-place solution},
  author={{NVIDIA Kaggle Grandmasters}}, year={2025}}
@article{hodel2024rearc,
  title={Addressing the Abstraction and Reasoning Corpus via
         Procedural Example Generation},
  author={Hodel, Michael}, journal={arXiv:2404.07353}, year={2024}}
```

References

[1] F. Chollet. On the Measure of Intelligence. arXiv:1911.01547, 2019.

[2] ARC Prize Foundation. ARC-AGI-2 Technical Report. 2025. `https://arcprize.org`.

[3] ARC Prize Foundation. ARC Prize 2025: Technical Report. arXiv:2601.10904, 2026.

[4] NVIDIA Kaggle Grandmasters (NVARC). ARC Prize 2025 (ARC-AGI-2) 1st-place solution. 2025.

[5] Qwen Team. Qwen3 Technical Report. 2025.

[6] M. Hodel. Addressing the Abstraction and Reasoning Corpus via Procedural Example Generation. arXiv:2404.07353, 2024.

[7] W. Lemes de Oliveira et al. ARC-AGI-2 Technical Report (LongT5-based system with test-time training). arXiv:2603.06590, 2026.